Tools for Data Quality Validation and Testing

In this train, we'll explore various tools available for data quality validation and testing. We'll discuss their features and how they can help in safeguarding data integrity and maintaining high data quality standards.

Learning Objectives

By the end of this train, you should be able to:

- Understand the tools available for data quality validation and testing.

Tools for Manual Testing

Manual testing can be the first step in the journey of having robust data quality tests in place. This involves running manual checks, for example, calculating means and missing values using built-in `Python`, `Spark`, or `Pandas` methods. This can help to familiarise you with your data to decide what type of constraints may be useful.

Pros

- It is the easiest to implement since there is no barrier of entry as you are already familiar with the toolset.
- It works from first principles, so you can implement exactly the tests you plan to do.
- This method can be useful in informing constraints for automated testing.

Cons

- This approach may be the most inefficient since it is not being optimised computationally.
- It may be time-consuming, and any new dataset will have to be implemented from scratch.
- You do not get defined metrics out of the box.

Tools for Automated Testing

Deequ

Deequ is a library that has been built on top of Apache Spark. Its purpose is to define "unit tests for data" so that you can measure the quality of large datasets. If using Deequ within a Python environment, there is a library called PyDeequ. There are four main components of Deequ:

- **Metrics Computations**:
  - `Profilers` leverage Analyzers to inspect the columns of your dataset.
  - `Analyzers` are foundational modules used to compute metrics for the profiling and validation of data at scale.
- **Constraint Suggestion**:
  - This is where the rules and constraints for the groups of Analyzers are defined. These are run over the datasets and returned to a Verification Suite.
- **Constraint Verification**:
  - Here data are validated based on the constraints that have been set.
- **Metrics Repository**:
  - The Metrics Repository allows you to save Deequ metrics and runs over time.

If we take a look at how these four components fit together into a flow, we will get something like this:

1. Analyzers are used to compute metrics for the data.
2. The Profiler is run on each column of the data to get a better understanding of the data.
3. The Constraint Suggestion step occurs where PyDeequ gives you a set of default constraints that it thinks will be useful. Here you would also modify the constraints to your specific use case as the defaults are likely to be very basic.
4. The Constraint Verification step happens when the constraints set in the previous step that make up the Verification Suite are validated. This allows you to see if the data quality passes or fails.
5. Once the Constraint Verification step has occurred, you can save the results of various runs to the Metrics Repository. This can be very helpful as you get a historical view of all the data quality checks that have been done on your data.

This whole process can be integrated within your ETL pipeline at multiple steps. You could have a system that kicks off to check the source data ingested as well as a system that ensures the transformations are occurring correctly.

Deequ has a rich set of checks that are available to choose from, and if you ever find yourself having to implement data quality testing with Deequ, you can find all the methods available here.

Some useful examples include defining the minimum and maximum allowed values for a column as well as checking for data completeness. You can even do checks for correlation between two columns using the `hasCorrelation()` or `hasMutualInformation()` methods.

> **To start using PyDeequ, all you need to do is run:**
> `pip install pydeequ`

For a closer look at how to use PyDeequ, take a look at the quick start.

Pros:

- This tool is specifically made for data quality testing and tracking.
- Seamless integration with Spark as it is built on top of Spark.
- Wide variety of built-in functions for data quality testing.
- Deequ provides a Profiler to automatically determine tests for your dataset using historical data.

Cons:

- Requires an Apache Spark environment (which is not really applicable to small datasets).

Great Expectations

Great Expectations is one of the leading tools that can be used for validating, documenting, and profiling your data.

> **Great Expectations can be installed using the following commands:**

```
pip install great_expectations
great_expectations init
```

The first command installs Great Expectations and the second command initialises your Great Expectations deployment for a new project. This means that Great Expectations will create a new directory with the following structure:

```
great_expectations/

great_expectations.yml
expectations
checkpoints
notebooks
plugins
.gitignore
uncommitted/
config_variables.yml
documentations
validations
```

The team from Great Expectations recommend deploying this setup within a virtual environment. Details on how to set up this virtual environment can be found here.

Great Expectations has the goal of helping you understand your data better so that communicating what you have built and expect becomes effortless. Three features are delivered to help you achieve this goal. Firstly, data are automatically profiled to build the Expectations. Next, Expectations are used to validate the data quality, and then this gets pushed automatically to Data Docs to give you an idea of your data's characteristics.

Let's clarify how this works in a bit more detail. What exactly is an Expectation? An Expectation is what you will use to define how your data should look. Expectations are built using Profilers which learn about your data. A Profiler is what Great Expectations uses to analyse your dataset and generate a first pass set of Expectations. Once the Expectations have been built, one can validate new data using the Expectations that have been generated. You also have the ability to update the Expectations that have been auto-generated to home in on your dataset's requirements. Data Docs use Expectations to describe your data and diagnose problems by giving you a visual report on which constraints have been met.

Pros:

- Auto-generated Expectations out of the box that can be modified as you learn about your data.
- High configurability (allows for very accurate studies of your data).
- Configurable data source connections.
- Can be run externally in a virtual environment, operating directly on the data and not requiring a Python or Spark environment.

Cons:

- Can be difficult to set up due to the high level of configurability.

Tool Comparison Summary

Feature	Manual Testing	Deequ	Great Expectations
Auto profiling	❌	✅ (Profiler)	✅ (Profiler)
Custom expectations	✅	✅	✅
Spark integration	❌	✅ (required)	❌ (optional)
GUI/visual output	❌	❌	✅ (Data Docs)
Learning curve	Low	Medium	Medium–High
Historical metrics	❌	✅ (Metrics Repository)	✅ (Validation Store)
Ease of integration	High (basic)	High (with Spark)	Medium (config required)

Conclusion

In conclusion, establishing robust data quality tests can begin with manual exploration and evolve into sophisticated automated solutions offered by tools like Deequ and Great Expectations, each with its own strengths and considerations.

▼Key Takeaways

- **Takeaway 1:** Manual testing, while less efficient for large datasets, provides an initial understanding of data and can inform the creation of constraints for automated testing.
- **Takeaway 2:** Deequ, built on Apache Spark, offers a scalable framework with built-in functions for data quality testing, profiling, and tracking metrics over time, making it ideal for big data environments.
- **Takeaway 3:** Great Expectations provides auto-generated and highly configurable data quality expectations, along with data profiling and documentation capabilities, offering flexibility and ease of understanding data characteristics.